sensors

MDPI

Article

Secured and Privacy-Preserving Multi-Authority Access Control System for Cloud-Based Healthcare Data Sharing

Reetu Gupta [1], Priyesh Kanungo [1], Nirmal Dagdee [2], Golla Madhu [3], Kshira Sagar Sahoo [4], N. Z. Jhanjhi [5,*], Mehedi Masud [6], Nabil Sharaf Almalki [7] and Mohammed A. AlZain [8]

1 School of Computer Science and Information Technology, Devi Ahilya Vishwavidyalaya, Indore 452001, India
2 SKITM College, Indore 452020, India
3 Department of Information Technology, VNR Vignana Jyothi Institute of Engineering and Technology, Hyderabad 500090, India
4 Department of CSE, SRM University, Amaravati 522240, India
5 School of Computer Science, SCS Taylors University, Subang Jaya 47500, Malaysia
6 Department of Computer Science, College of Computers and Information Technology, Taif University, Taif 21944, Saudi Arabia
7 Department of Special Education, College of Education, King Saud University, Riyadh 145111, Saudi Arabia
8 Department of Information Technology, College of Computers and Information Technology, Taif University, Taif 21944, Saudi Arabia
* Correspondence: noorzaman.jhanjhi@taylors.edu.my

Abstract: With continuous advancements in Internet technology and the increased use of cryptographic techniques, the cloud has become the obvious choice for data sharing. Generally, the data are outsourced to cloud storage servers in encrypted form. Access control methods can be used on encrypted outsourced data to facilitate and regulate access. Multi-authority attribute-based encryption is a propitious technique to control who can access encrypted data in inter-domain applications such as sharing data between organizations, sharing data in healthcare, etc. The data owner may require the flexibility to share the data with known and unknown users. The known or closed-domain users may be internal employees of the organization, and unknown or open-domain users may be outside agencies, third-party users, etc. In the case of closed-domain users, the data owner becomes the key issuing authority, and in the case of open-domain users, various established attribute authorities perform the task of key issuance. Privacy preservation is also a crucial requirement in cloud-based data-sharing systems. This work proposes the SP-MAACS scheme, a secure and privacy-preserving multi-authority access control system for cloud-based healthcare data sharing. Both open and closed domain users are considered, and policy privacy is ensured by only disclosing the names of policy attributes. The values of the attributes are kept hidden. Characteristic comparison with similar existing schemes shows that our scheme simultaneously provides features such as multi-authority setting, expressive and flexible access policy structure, privacy preservation, and scalability. The performance analysis carried out by us shows that the decryption cost is reasonable enough. Furthermore, the scheme is demonstrated to be adaptively secure under the standard model.

Keywords: electronic health records; access control; cloud storage; attribute-based encryption; multiple authorities; privacy preservation

check for updates

Citation: Gupta, R.; Kanungo, P.; Dagdee, N.; Madhu, G.; Sahoo, K.S.; Jhanjhi, N.Z.; Masud, M.; Almalki, N.S.; AlZain, M.A. Secured and Privacy-Preserving Multi-Authority Access Control System for Cloud-Based Healthcare Data Sharing. *Sensors* **2023**, *23*, 2617. https://doi.org/10.3390/s23052617

Academic Editor: Paul Davidsson

Received: 28 November 2022
Revised: 14 February 2023
Accepted: 21 February 2023
Published: 27 February 2023

Copyright: © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The modern health industry is adopting Internet of Things (IoT) technology for providing advanced healthcare services [1]. A wide range of IoT devices and applications are designed for healthcare needs, e.g., sensors, remote healthcare monitoring applications, telemedicine consultation applications, etc. Healthcare organizations can collect, record, and monitor patient data regularly, providing them with adequate treatment in every situation. Patients can be treated well in emergencies by making use of their electronic

health records (EHRs). For example, let us assume that a patient is suffering from chronic heart disease and they use body sensors to record their blood pressure. As patient monitoring services make their EHR available online, their doctors can treat them efficiently. In addition, in some unfavorable conditions when they may need medical attention, any treating doctor can access their data to make the right diagnosis. The availability of their EHRs can save their life.

Patients can maintain their EHR data on cloud storage servers and receive various advantages such as 24/7 access, easy management, and tracking and sharing of the EHR data for treatment as well as beneficiary purposes. However, cloud-based data outsourcing is not trusted because the data owners lose physical control of their data. Performing data encryption before uploading to the cloud is one solution to the above problem. This limits accessibility for legitimate users, as they will need decryption keys for accessing the data. Data owners can only issue the keys to known users. Other legitimate users cannot access the encrypted data in this case. Therefore, there is a need for a secure data-sharing system allowing the data owner to provide access control to data in a fine-grained manner. The system should have proper attributes and key management facilities to serve all legitimate users. To accomplish this outsourcing paradigm, the cloud-based EHR approach is motivated by the concepts of "Availability, Scalability, Cost efficiency, and Convenience" [2]. Dissemination of patient data through cloud-based EHRs is advantageous, but it must be performed with such care that the privacy of patients is protected.

Various cryptographic access control models have been defined to provide data security and control unauthorized data access. For this need, attribute-based encryption, also known as ABE [3], seems to be a suitable option. It allows for fine-grained and flexible access control. One type of ABE called ciphertext-policy attribute-based encryption [4] or CP-ABE is a technique in which the user's secret key is generated for their attributes and an access policy is coupled with ciphertext. Access policies are typically expressed by threshold gates or AND-OR gates over the attributes. Users receive their attribute keys in terms of secret keys issued by the attribute authorities. CP-ABE is a kind of public key encryption (PKE) technique. It provides access control for a large group of users because it allows decryption only when the user carries the attribute set that matches the policy. Credentials or attributes are issued to the users by various attribute authorities, which may be from single or multiple domains. Several ABE techniques for cloud-based data sharing are given [5–8]. However, these schemes consider a single domain authority that manages all the attributes. In practical scenarios, users from multiple domains may request a download of the shared data and different administrating authorities may issue the attributes. Additionally, normal cipher-policy ABE schemes [4,9] store the access policy with outsourced encrypted data, but in plain text. This may result in the disclosure of sensitive information regarding the encrypted data. Let us understand cloud-based data sharing with the help of a healthcare use case.

1.1. Use-Case

In the healthcare system, various users such as doctors, nurses, research scholars, insurance agents, etc., may need to access the data. In this case, various authorities, e.g., hospitals, clinical research centers, insurance companies, and pharmaceutical companies, may issue various attribute keys to the concerned users (as shown in Figure 1). The shared EHRs of the patients may be protected by instituting access policies, such as any user who possesses the "Research Assistant" attribute in the domain of renowned "Research Organization" or "Insurance agent" attribute in the domain of "Insurance Company" from where the patient is receiving services being permitted to access the data. Therefore, in healthcare industries, where the user fraternity is a large group, multi-authority CP-ABE schemes are a suitable option for facilitating efficient and secure EHR sharing. All the above-stated users in the healthcare system come from the open domain, i.e., they are not known to the data owner.

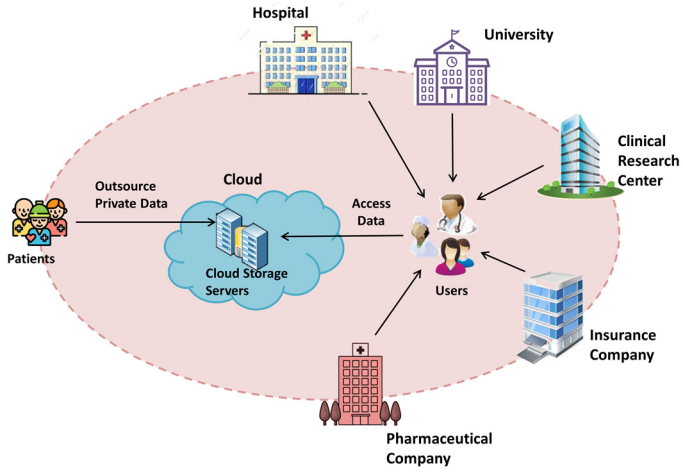


Figure 1. Cloud-based healthcare data sharing.

To control access to shared data, attributes issued by various open-domain attribute authorities are used in access policies. Another important issue that must be considered in EHR data sharing is to give access to closed-domain users, i.e., the users known to the data owner, e.g., family friends, relatives, etc. Flexibility in handling the access requirements of both closed- and open-domain users makes the sharing of health data more practical and effective [10]. In emergencies, family friends or relatives may access the data. The privacy of the patient is also crucial in this kind of data sharing. In the above instance, if a patient outsources their medical data with policy "[(Profession = "Doctor" and Specialization = "Cardiology" and Affiliation = "AIIMS") or (Profession = "Doctor" and Specialization = "Cardiology" and Research Associate = "University Hospital") or (Relation = "Friend")]", then everyone including adversaries and the cloud service provider (CSP) can look at the policy formulation and figure out that the shared data are of a patient who is suffering from heart disease. This results in privacy leakage even though the ciphertext of EHR data is protected well. Therefore, it is essential to keep the access policy a secret to protect sensitive data.

1.2. Related Works

Numerous researchers have discussed data outsourcing in the cloud environment. Attribute-based encryption introduced by Sahai-Waters [11] is considered as most prominent scheme to institute access control for encrypted data. Although ABE and similar systems [4,9,11,12] employ one-to-many encryption concepts, there are some concerns with these techniques. The single authority managing the key issuance process for all the eligible users may decrypt every ciphertext by using issued secret keys. This is called the key escrow problem. Another problem in these schemes is low system performance due to over-reliance on a single authority for handling all of the system's keys. This motivated the concept of establishing multiple authorities for key management tasks in a distributed manner. Several multi-authority ABE schemes and schemes supporting policy privacy have been presented in the literature.

1.2.1. Multi-Authority ABE Schemes

Multi-authority ABE was first discussed in the work [13], where only one CA, also known as the central authority, and numerous AAs, also known as attribute authorities, controlled key management. The CA and AAs were responsible for issuing keys for identity and keys for attributes, respectively. The use of the global identifier GID prevented user collusion problems. However, the CA was capable of decrypting any ciphertext. Chase and Chow [14] improved their scheme (CC-MA-ABE) by removing the CA and introducing an anonymous key distribution mechanism with the help of pseudo-random functions. Both schemes in [13,14] had the limitation that they supported an AND policy architecture only. Liu et al. [15] presented the MACP-ABE scheme, a fully secure scheme in

the standard model where multiple central and attribute authorities collaborate to work together. The central authorities are responsible for issuing keys related to the user's identity, and the attribute authorities control the issuance of the attribute-related keys. Lewko and Waters [16] designed a decentralized CP-ABE and demonstrated security under the random oracle model. The linear secret sharing scheme, also known as LSSS, was used for specifying the access policy. At setup and key generation, no coordination between authorities was necessary, nor was there a central authority. Li et al. [17] proposed a scheme where the user's community can be divided into public and personal domains (PUD and PSD, respectively) depending upon their role in the system. Their scheme was based on scheme [14] and policy was specified using a conjunctive normal form (CNF) structure. Ibraimi et al. [18] suggested a scheme for patient health record sharing in a multi-authority (two authorities) setting. They introduced social domains and professional domains for different authorized users. Several pieces were proposed to cater to different issues regarding MA-ABE. Ruj et al. [19] addressed the revocation function in a multi-authority setting. However, their method had heavy communication overhead and key update computation overhead. The authors of [20] proposed cipher policy ABE schemes for supporting multi-authority scenarios and handling user revocation features. To improve efficiency, the decryption process was outsourced. AA became a bottleneck in the scheme, as it had to calculate update keys for each unrevoked user. The scheme presented in [21] improved the CP-ABE scheme for secure PHR sharing [22] for the multi-authority scenario. The authors also defined public and private domains for their PHR-sharing scheme. Li et al. [23] introduced an access control scheme for cloud storage supporting decryption outsourcing. The scheme was also a multi-authority scheme and was evidenced to be adaptive and secure. In order to eliminate key escrow and minimize computation and communication costs, Hu et al. [24] provided an MA-ABE scheme that resisted key escrow and had a ciphertext of constant size. Furthermore, in the MA-KPABE scheme presented in [25], verifiability of partial decryption ciphertext (PDC) and delegation features were added. Ma et al. [26] presented two decentralized CP-ABE techniques for the standard model. The CAs and AAs operated independently of one another. The first technique was constructed using a group of composite orders. The second technique produced ciphertexts of constant size in the groups of prime order. The first technique worked for any monotonic structure, while the second worked for AND-gate policies.

1.2.2. Policy Preservation in Attribute-Based Encryption

Because encrypted data on cloud storage servers remained in plaintext form when outsourced, access policies were shared with different users in all of the above schemes. This may cause potential exposure of sensitive information about the data owner as well as the consumers of the data. Several works [27,28] introduced CP-ABE schemes with partially hidden access control policies to protect the disclosure of this sensitive information. The access policies in these schemes divide attributes into two parts, i.e., attribute name and attribute value. In a partially hidden access policy, the attribute value that reveals sensitive information is made hidden, e.g., ([(Profession = "*" and Specialization = "*" and Affiliation = "*") or (Profession = "*" and Specialization = "*" and Research Associate = "*") or (Relation = "*"))). Li et al. [29] used the anonymous key issuing protocol used in CC-MA-ABE [14] and proposed an accountable multi-authority CP-ABE. Han et al. [30] developed a decentralized CP-ABE (PPDCP-ABE) scheme to eliminate dependency on and trust in central authorities while maintaining the privacy of users. In their scheme, multiple authorities may operate autonomously. The Pedersen commitment protocol [31] and zero-knowledge-proof protocol [32] were used to protect the attributes' privacy. In [32], a policy-hiding CP-ABE scheme was proposed to improve decryption efficiency. The authors pioneered the "match-then-decrypt" method, in which ciphertext components were routed to the decryption test. Without performing the actual decryption, it checked the satisfiability of the hidden attributes policy for the attribute private key. Chen et al. [33] designed a privacy-preserving decentralized CP-ABE where the secret key was taken out

with privacy. Their proposal required no central AA or multi-authority collaboration. They used the scheme [31] proposed by Pederson and oblivious attribute certificates [34]. The users receive secret keys for legitimate identity attributes, but AAs cannot find any useful information. Zhong et al. [35] proposed an access control scheme with a hidden policy on multi-authority architecture. To hide policy, attributes were obfuscated using a one-way anonymous key agreement protocol. Yang et al. [36] proposed a method for controlling Big Data access. Instead of hiding the attribute values, they hid the whole attribute for privacy purposes. They utilized an attribute bloom filter, which detects an attribute and its precise location in the access policy. Ying et al. [37] presented a lightweight policy-preserving CP-ABE scheme for EHR sharing on the cloud. The access policy was fully hidden by use of the attribute cuckoo filter (ACF). PASH, proposed in [38], was designed to provide access control for smart health. The CP-ABE scheme supported a large universe and used partially hidden policies. It also handled decryption tests efficiently and provided full security. Yan et al. [39] introduced a multi-authority ABE with privacy preservation and dynamic policy updating. This scheme was suggested for a multi-authority scenario, but it could not prevent malicious users from sharing their private keys. Belguith et al. [40] proposed PHOABE for cloud-assisted IoT, where multiple authorities were considered and a fully hidden policy was maintained by obfuscating the attributes. Their scheme was based on the scheme in [16]. It introduced a semi-trusted cloud server for outsourcing the heavy decryption process. This minimized computation overhead on resource-constrained devices. Zhang et al. [41] developed a hidden ciphertext-policy scheme for a large universe and proposed an efficient decryption procedure. Chinnasamy et al. [42] proposed a policy-hidden CP-ABE scheme for providing access control in an IoT environment. The SHA1 hashing algorithm was used for policy anonymization. Research works in [43,44] presented challenges related to cloud storage resources and applications in IoT. Najafi et al. [45] introduced a system with attribute privacy and search capabilities over encrypted data. In order to keep medical records safe and accessible, they created a storage and retrieval system. The approach was safe against keyword guess attacks in the standard model.

Analysis of the aforementioned schemes elucidated some comparative notes and observations. The following is a brief outline of issues with prominent MA-ABE schemes.

(1) Several attribute-based access control schemes [13–23,26] in the multi-authority domain have been proposed, but these methods lack the policy preservation aspect;
(2) There are schemes supporting privacy preservation approaches [27–30,32,33,35–41,46], but some of them are single-authority [27,38] and some of them use fully hidden access policies [35,40], which are more rigid in nature;
(3) The security of most of the multi-authority schemes was validated in weaker security models, i.e., selective security [35,39], where adversaries need to declare a challenged access policy structure before obtaining the public parameters. There is a requirement for higher security in the above scenario.

1.3. Our Contributions

To resolve the aforementioned problems, SP-MAACS, a secure and privacy-preserving multi-authority access control system for cloud-based data sharing is proposed. Our SP-MAACS is a secure MA-CP-ABE scheme with a partial policy hiding feature. Two components make up an attribute: the name and the value. Concrete attribute values are used in the access policy. They are encoded in the ciphertext components. The access policy in plaintext is also saved with the encrypted data. It includes attribute names, not values. In our scheme, we used the access policy in DNF form, also called "alternative routes to authorization" [47]. A set of satisfiable sub-policies can be derived from the main policy. For example, a policy for the data D represented as an arbitrary logical formula such as [a1 $\wedge$ (a2 $\vee$ a3)] can be written as [(a1 $\wedge$ a2) $\vee$ (a1 $\wedge$ a3)]. Here the set of sub-policies is [(a1 $\wedge$ a2), (a1 $\wedge$ a3)]. Generally, when a data owner decides on an access policy, he starts framing it using a combination of alternatives. This standard method of specifying access

policy corresponds to the DNF structure employed by our approach. The following is a concise summary of the primary features of our system:

→ The proposed scheme incorporates the important aspect of privacy preservation in a multi-authority setting. Along with this added-on feature of privacy preservation, our multi-authority access control scheme also achieves better decryption efficiency;
→ The scheme is designed to support open- and closed-domain users and allows for employing fine-grained access control. The access policy formulated using DNF makes the policy specification more flexible and expressive. As our system is scalable, it allows users from varied domains and makes it better suited for real-world applications;
→ The scheme is adaptively secure. It achieves resistance to collusion attacks, as the users cannot integrate their attributes to access shared data. The scheme is demonstrated as secure in the standard model.

1.4. Organization

The rest of the paper is organized into the following sections: Section 2 compares the traits of prominent schemes studied in the previous section and our scheme. In Section 3, we outline some standard cryptographic definitions and access structure definitions. In Section 4, we propose the system model, the definitions of algorithms, and the security model. The SP-MAACS system construction is illustrated in Section 5. Section 6 discusses the scheme's security, performance analysis, and implementation results. Section 7 concludes the research work.

2. Characteristics Comparison

Table 1 provides a comprehensive comparison of some major characteristics of prominent CP-ABE schemes and SP-MAACS. The comparison involves important features such as multi-authority setting, access policy structure and its expressiveness, privacy preservation, and the security settings of the schemes. The schemes [25,38,39,48] were designed for healthcare systems. From this comparison, we can see that the adaptively secure privacy-preserving schemes [28,38,48] are single-authority schemes. The multi-authority CP-ABE schemes [16,23,25,26] are adaptively secure but do not offer privacy preservation of the access policy. The schemes [35,39] are multi-authority and privacy-preserving schemes, but their security is only evidenced in weaker selective models. As far as our survey goes, the SP-MAACS scheme is the only one that preserves privacy while also offering adaptive security in a multi-authority setting.

Table 1. Prominent ABE schemes vs. SP-MAACS scheme.

Scheme	CP/KP	Multi-Authority	Privacy-Aware	Expressiveness	Security	Group Order	Security Model
Single-Authority Privacy Preserving Schemes with Adaptive Security in Standard Model							
[28]	CP	×	√ (Partially hidden policy)	LSSS	Adaptive	Composite	Standard
[38]	CP	×	√ (Partially hidden policy)	LSSS	Adaptive	Composite	Standard
[48]	CP	×	√ (Partially hidden policy)	LSSS	Adaptive	Composite	Standard

Table 1. *Cont.*

Scheme	CP/KP	Multi-Authority	Privacy-Aware	Expressiveness	Security	Group Order	Security Model
Multi-Authority Schemes							
[16]	CP	√	×	LSSS	Adaptive	Composite	Random oracle
[23]	CP	√	×	LSSS	Adaptive	Composite	Standard
[26]	CP	√	×	LSSS	Adaptive	Composite	Standard
[25]	KP	√	×	LSSS	Adaptive	Composite	Standard
Multi-Authority Privacy Preserving Schemes							
[35]	CP	√	√ (Fully hidden policy)	LSSS	Selective	Prime	Random oracle
[39]	CP	√	√ (Partially hidden policy)	LSSS	Selective	Prime	Standard
Multi-Authority Privacy Preserving (Partially Hidden policy) Scheme with Adaptive Security in Standard Model							
Ours	CP	√	√ (Partially hidden policy)	LSSS	Adaptive	Composite	Standard

3. Mathematical Preliminaries

This section introduces the formal definitions and notations of the proposed scheme:

3.1. Composite Order Bilinear Groups

The authors defined composite order bilinear groups in [49].

Definition 1. *The order O of a bilinear group is defined as* $O = p_1p_2p_3$*, i.e., the product of three different primes (here* p_1*,* p_2 *and* p_3*). Let G and* G_T *be cyclic groups of the order O. Let the subgroup in G with the order* p_i *be denoted as* G_{p_i}*.* g_{p_1}, g_{p_2} *and* g_{p_3} *are generators of* G_{p_1}*,* G_{p_2} *and* G_{p_3}*, respectively. Let* $e : G\ X\ G \rightarrow G_T$ *be the mapping and the following be the required properties:*

1. *Bilinear property:* $\forall\ c, d \in G$ *and* $a, b \in ZN$*.* $e\left(c^a,\ d^b\right) = e(c,\ d)^{ab}$*.*
2. *Property of non-degeneracy:* $\exists\}\ \in G$*, where* $e(g,\ g)$ *in* G_T *is of the order O.*
3. *Computability:* $\exists$ *an algorithm to compute* $e(c, d)\ \forall c, d \in G$ *efficiently.*
4. *Orthogonality:* $e\left(g_{p_1}, g_{p_2}\right) = 1$ *for any* $g_{p_1} \in G_{p_1}$ *and any* $g_{p_2} \in G_{p_2}$*.*

3.2. Access Structure for Privacy Preservation

Here we first define normal access structure.

Definition 2. *Let us name the universe of attributes AU. An access structure* φ *on AU is a collection of non-empty attribute sets, i.e.,* $\varphi \subseteq 2^{AU} \backslash \varnothing$*. The collection of attribute sets, which is present in* φ*, is called the authorized set; all the other attributes lie in an unauthorized set. In addition, an access structure is called monotonic if* $\forall C, D$*: if* $C \in \varphi$ *and* $C \subseteq D$*, then* $D \in \varphi$*.*

Now let us define linear secret sharing scheme (LSSS).

Definition 3. *Let* Π *be a secret-sharing scheme in which:*

(1) The generated share for each participant is a vector over Z_p*.*

(2) $\exists$ *A matrix W of m rows and n columns, where* $\forall$*rows* $\in W$*, the* j^{th} *row is marked with the function* $\rho(j)$*; then it is called a linear scheme. Secret s is randomly chosen such that* $s \in Z_p$ *and a vector is formed so that* $v = (s,\ v_2\ \ldots\ v_n) \in Z_p^n$*. Now let us take* $\lambda = W\ v^T$ *such that share* λ_j *is for participant* $\rho(j)$ *so we can write* $\lambda_j = W_j \cdot v$*.*

(3) *Linear reconstruction property: Let us denote S as an authorized set and take I = {j: ρ(j)∈ S}. For an LSSS scheme, there exists a constant set* $\left(\mu_j \in Z_p\right)_{j\in I}$*, used to compute the secret s:* $\sum_{j\in I} \mu_j \lambda_j = s$.

Access Structure for privacy preservation:

Definition 4. *Let us take an access structure φ = (W, ρ, Z) for describing an access policy. W is a share-generating matrix with the dimensions l by n that is connected with a secret sharing scheme, ρ is a map function and* $Z = \left(Z_{\rho(j)}\right)_{1\leq j\leq l}$ *is a set of corresponding possible values of the respective attribute. Function ρ maps each row of W to the name of the attribute present in the access policy. We are keeping Z hidden in our scheme and the share-generating matrix W and function ρ are attached to the ciphertext.*

Disjunctive Normal Form

Definition 5. *In discrete mathematics, a canonical normal form of a Boolean formula can be written as OR of ANDs. It is termed the sum of products (SOP). This normal form is called a disjunctive normal form (DNF). It can be written as* $A = A_1\ V\ A_2\ V\ \ldots\ VA_n\ \ (n \geq 1)$*, where* $A_1\ ,\ A_2\ ,\ldots,\ A_n$ *are called sub-formulas and they are all conjunctions of the terms.*

4. System Model, Algorithms, and Security Model

This section discusses the system model, various algorithms designed for the scheme, and the security model of the SP-MAACS scheme.

4.1. System Model

Figure 2 shows the major entities in our SP-MAACS system for cloud-based data sharing. The entities are:

(1) ***Data Owner (DO):*** The data owner decides on an access policy and formulates it using the attributes present in the attribute universe. Then they encrypt the data under this policy. These encrypted data are stored on the cloud servers, but the access policy is kept partially hidden (Steps 1 and 2 in Figure 2);

(2) ***Central Authority (CA):*** The responsibilities of a CA can be defined as: (1) Generate global public parameters for the system. (2) Service the user's request for registration and issue identity keys based on their global identifier (gid) (Step 3 in Figure 2);

(3) ***Attribute Authorities (AAs):*** The responsibilities of an AA can be defined as: (1) Generate public keys for the attributes they manage. Each AA may have the authority to issue any number of attributes, but a single AA is authorized to issue each attribute. (2) Verify the user's possession of the attribute and issue a secret key for the user's attributes;

(4) ***Cloud Service Provider (CSP):*** The CSP essentially acts as a resource provider in place of the cloud, replicating that role for the cloud. The data owners use its data storage service and the users send a query for required data to access it. Furthermore, there is an assumption that the CSP is curious about obtaining the knowledge of data, but at the same time, it is honest;

(5) ***User:*** A unique global identity is allotted to every user. They receive a secret key issued for numerous attributes from the responsible AA. The user sends the request for data access to the CSP along with their acquired secret keys, and if the attributes possessed by them are required in satisfying the access policy, they can obtain the data (Steps 5 and 6 in Figure 2).

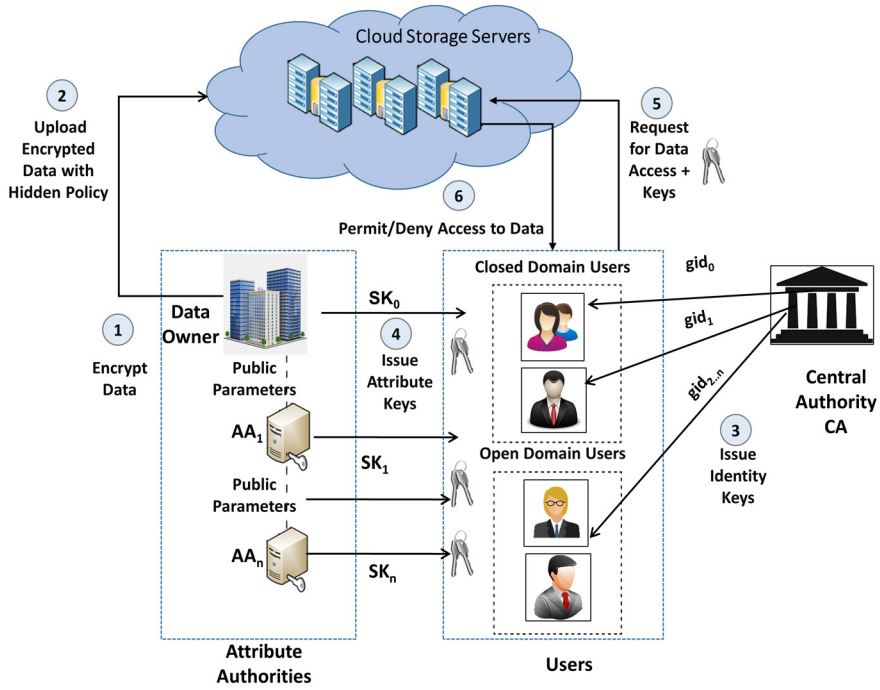


Figure 2. System model for SP-MAACS scheme.

In the SP-MAACS scheme, attribute authorities in the open domain can be hospitals, clinical research centers, etc. These authorities issue attributes to users such as doctors, nurses, etc. The EHR data owner plays the role of an AA for issuing secret keys to the users in a closed domain, e.g., friends, relatives, etc. (Step 4 in Figure 2).

4.2. Algorithms

The SP-MAACS scheme uses the following four algorithms in its construction.

4.2.1. System Initialization

GlobalSetup(λ) $\rightarrow$ GPP: The algorithm uses the input λ which is also called the security parameter. After the setup executes, global public parameters GPP are generated.

CASetup(GPP) $\rightarrow$ (MPK, MSK): The central authority(CA) executes the CASetup algorithm. It outputs public key MPK and secret key MSK. All the authorities use MPK for verification purposes.

AASetup(GPP, k, U_k) $\rightarrow$ (APK_k, ASK_k): Every authority AA_k present in the system executes this algorithm, where the inputs are GPP and its attribute domain is called U_k. No two authorities have a common attribute domain, which means for i $\neq$ j, $U_i \cap U_j = \varnothing$. This algorithm produces public key APK_k and secret key ASK_k.

4.2.2. Encryption

Encrypt(K, ψ, GPP, $\cup$ APK_k) $\rightarrow$ (CT): This algorithm takes as input the GPPs, a symmetric key K by which data are encrypted, an access policy structure ψ, and a collection of public keys of applicable authorities. It generates the ciphertext.

4.2.3. User Key Generation

CAKeyGen(GPP,gid) $\rightarrow$ ($CAPK_{gid}$, $CASK_{gid}$): The user submits their gid as input to this algorithm. Taking GPP as another input, it produces the gid-related identity-key $CASK_{gid}$, which is held by the user. The public key $CAPK_{gid}$ is given to the AAs for generating attribute-related keys.

AAKeyGen($S_{gid,k}$,)GPP, MPK, $CAPK_{gid}$, ASK_k) $\rightarrow$ ($ASK_{S,gid,k}$): When a user requests k^{th} authority for generating keys for an attribute set $S_{gid,k}$, AA_k runs this algorithm with

inputs as $S_{gid,k}$, GPP, MPK, $CAPK_{gid}$, and ASK_k. If $CAPK_{gid}$ is invalid, then it returns $\perp$, else it returns corresponding attribute-related keys $ASK_{S,gid,k}$ for attribute set $S_{gid,k}$.

4.2.4. Decryption

Decrypt(CT, GPP, FK_{gid}) $\rightarrow$ (K): The decryption algorithm uses a global public parameter, the ciphertext denoted as CT, and the final secret keys' set FK_{gid} as the inputs. The decryption is complete when the user's attributes satisfy the policy.

4.3. Security Model

We define the security model for SP-MAACS through a security game between adversary A and challenger C. We assumed that A can corrupt at most K-1 AAs. Let K_c denote the index set of corrupted AAs and K_{uc} denote the index set of uncorrupted AAs, where $K_{uc} = K \backslash K_c$. The steps of the game are as follows:

Setup: C executes the algorithms GlobalSetup, CASetup, and AASetup and transfers the GPP, MPK, and $\cup_{k=1}^{K} APK_k$ to the adversary A. Assume the adversary corrupts K_c AAs such that $K \backslash K_c \neq \Phi$. The challenger C passes the secret key $\{ASK_k \mid k \in K_c$ to A.

Phase 1: Adversary A can obtain the secret keys for the AAs who have been corrupted.

CAKey queries: For these queries, challenger C responds by $CAPK_{gid}$ and $CASK_{gid}$.

AAKey queries: For the attribute set, the adversary submits $S_{gid,k}$ and $CAPK_{gid}$ to C, where $k \in K_{uc}$. C returns the $\left\{ ASK_{S,gid,k} \right\}_{k \in K_{uc}}$.

Challenge: If adversary A finds that phase 1 is complete, it sends to C two messages M_0, M_1, which are equal in length, and two challenge access structures $\psi_1^* = (W_1, \rho_1, Z_1)$ and $\psi_2^* = (W_2, \rho_2, Z_2)$. Here the condition is that ψ_1^* and ψ_2^* cannot be satisfied by any attribute key query performed in phase 1. A random coin $c \in \{0, 1\}$ is flipped by C; then, it sets $CT_{\psi_c}^* \leftarrow Encrypt(K_c, \psi_c, GPP, \cup APK_k)$ and passes this challenged ciphertext to A.

Phase 2: As in phase 1, adversary A can again obtain adaptive secret key queries.

Guess: Adversary A gives its guess c′ of c as output and wins the game if $c' = c$. Probability *Pr* of being $c' = c$, i.e., $\left| Pr[c' = c] = \frac{1}{2} \right|$ is called the advantage of adversary A. To compute *Pr*, A and C choose random bits.

Definition 6. *Our SP-MAACS scheme with a privacy-preserving feature is fully secure since no probabilistic polynomial-time (PPT) adversary has a non-negligible advantage in the above game.*

5. Scheme Construction

The following is the detailed construction of SP-MAACS:

5.1. System Initialization

The following three algorithms are used for system setup:

GlobalSetup(λ) $\rightarrow$ GPP: The algorithm is called to initialize the system. It uses the input λ and generates GPP (global public parameters). Then, it chooses two bilinear groups G and G_T of order $O = p_1 p_2 p_3$ and mapping $e : G \; X \; G \rightarrow G_T$. Let the subgroup in G with the order $p_i b$be denoted as G_{p_i}. g is a randomly chosen element from G_{p_1}. I_3 is a generator of G_{p_3}. The GPP are made available as = (O, e, g, I_3, Ω_{Sign}), where Ω_{Sign}= (GenKey,Sign,Verify) is a secure signature scheme and will be used to counter any collusion attempts.

CASetup(GPP) $\rightarrow$ (MPK, MSK): The CA runs the GenKey algorithm of Ω_{Sign} to obtain signature key MSK and verification key MPK. All the AAs use MPK.

AASetup(GPP, k, Uk) $\rightarrow$ (APK_k, ASK_k): The attribute authority AA_k runs the algorithm. Let us assume attribute universe AU_k consists of n attribute names e.g., (a_1, a_2, . . . , a_n), and each attribute a_i has n_i attribute values, e.g., $(a_{i,1}, a_{i,2}, \ldots, a_{i,n_i})$. Each AA_k chooses

a random exponent $t_{k,n_i} \in Z_N$ and computes $T_{k,n_i} = g^{t_{k,n_i}}$. It also chooses two random exponents $\alpha_k,\ \beta_k \in Z_N$ and computes the public keys of AA_k as:

$$PK_{k,1} = e(g,g)^{\alpha_k} \tag{1}$$

$$PK_{k,2} = g^{\beta_k} \tag{2}$$

Hence, the cumulative public key of AA_k is:

$$APK_k = \left\{ e(g,g)^{\alpha_k}, g^{\beta_k}, g^{t_{k,n_i}} \forall a_{k,n_i} \in U_k \right\} \tag{3}$$

And the cumulative secret key of AA_k is:

$$ASK_k = \left\{ \alpha_k, \beta_k, t_{k,n_i},\ \forall a_{k,n_i}\ \in U_k \right\} \tag{4}$$

5.2. Encryption

When the data owner outsources EHR data, they use a symmetric encryption algorithm and a key K to encrypt the data. Then, they encrypt key K using the following encrypt algorithm with the access policy ψ. Encryption is performed as follows:

Encrypt(K, ψ, GPP,$\cup APK_k$) $\rightarrow$ (CT): As we mentioned to choose the DNF of a set of sub-policies, let us assume there are q sub-policies, i.e., $\{\psi_i\}_{i=1,2..,q}$. For simplicity, let us call each of them *W*.The sub-policy *W* is the LSSS matrix. Let us take some rows and columns in the LSSS matrix as land n. The function ρ associates each row W_j of *W* to attribute $\rho(j)$. To make the policy hidden, a set of attribute values is denoted as $Z = (Z_{\rho(1)},\ \ldots\ Z_{\rho(l)})$ and is attached to the access policy. Thus, the sub-policy can be expressed by $(W,\ \rho,\ Z)$. For each sub-policy, the following steps are run:

Step 1. The data owner chooses $s \in Z_p$ and a random vector

$$v = (s,\ v_2\ \ldots\ v_n)^T \in\ Z_N^n$$

Step 2. For each row of the matrix, the following is calculated:

$$\lambda x = W_x \cdot v$$

Step 3. Then, for every $x\ \epsilon\ [l]$, it selects a random exponent $r_x\ \epsilon Z_p$ and calculates:

$$C = K \cdot \left(\prod_{k \in SAA} e(g,g)^{\alpha_k} \right) s = K \cdot e(g,g)^{s \sum_{k \in SAA} \alpha_k} \tag{5}$$

$$C_0 = g^s \tag{6}$$

$$C_{1,x} = g^{\sum_{k \in SAA} \beta_k \lambda x} \cdot \left(T_{\rho(x)}^{Z_{\rho(x)}} \right)^{-r_x} \tag{7}$$

$$C_{2,x} = g^{r_x} \tag{8}$$

Here, SAA is defined as the index set of AA_k. This set consists of an index of AAs whose attributes are present in the policy.

Step 4. Finally, the data owner sends the ciphertext data to a cloud E(K) = $\{\{\psi_i\}_{i=1,2,3,\ldots,q}$, $\{E_i\}_{i=1,2,3,\ldots,q}\}$, where, $E_i = \left\{ C,\ C_0,\ (C_{1,x},\ C_{2,x})_{x \in \{1,\ldots,l\}} \right\}$.

The decryption of the data by the user is possible when their attribute set matches any of the sub-policy $(W,\ \rho,\ Z)$. The matching process is successful when for all $x \in \{1,\ \ldots,\ l\}$, $a_{\rho(x)} = Z_{\rho(x)}$ and constants $\mu_x \in Z_N$ such that $\sum_{x \in l} \mu_x \lambda_x = s$.

5.3. User Key Generation

Every new user receives a unique gid after he registers himself in the system. For obtaining identity-related keys, he requests CA. The CA runs the CAKeyGen algorithm.

After that, the user applies attribute-related keys, and different AAs run the AAKeyGen algorithm for this.

CAKeyGen(GPP,gid) $\rightarrow$ ($CAPK_{gid}$,$\mathrm{CASK_{gid}}$): The CA demands the user's gid for issuing identity keys. CA randomly chooses $r_{gid} \in Z_N$ and $\mathrm{R_{gid}} \in G_{p_3}$, then sets $\mathrm{CASK_{gid}} = g^{r_{gid}}\mathrm{R_{gid}}$. Then, it uses MSK to compute $\gamma_{gid} = Sign\left(MSK,\ gid \parallel \mathrm{CASK_{gid}}\right)$. Finally, it sends $CAPK_{gid} = \left(gid, \mathrm{CASK_{gid}}, \gamma_{gid}\right)$ and $\mathrm{CASK_{gid}}$ to the user.

AAKeyGen($S_{gid,k}GPP,\ MPK, CAPK_{gid}, ASK_k$) $\rightarrow$ ($\mathrm{ASK_{S,gid,k}}$): To obtain the attribute-related keys issued, a user passes their attribute set $\mathrm{S_{gid,k}}$ to the $\mathrm{AA_k}$, which belongs to their domain. Then, the $\mathrm{AA_k}$ uses the MPK to verify whether the $\mathrm{CAPK_{gid}}$ is valid. If valid, for issuing keys related to $\mathrm{S_{gid,k}}$, $\mathrm{AA_k}$ randomly selects $\mathrm{R_{gid,k}} \in G_{p_3}$ and computes $\mathrm{SK_{gid,k}} = g^{\alpha_k}\mathrm{CASK_{gid}}^{\beta_k}\mathrm{R_{gid,k}} = g^{\alpha_k}g^{r_{gid}\beta_k}\mathrm{R'_{gid,k}}$, where $\mathrm{R'_{gid,k}} = \mathrm{R_{gid}}^{\beta_k}\mathrm{R_{gid,k}}$, else it aborts.

For each attribute $\mathrm{i} \in \mathrm{S_{gid,k}}$, it randomly selects $\mathrm{R_{gid,k,i}} \in G_{p_3}$ and computes $\mathrm{SK_{gid,k,i}} = \mathrm{CASK_{gid}}^{t_{k,ni}}\mathrm{R_{gid,k,i}} = {T_{k,ni}}^{r_{gid}}\mathrm{R'_{gid,k,i}}$, where $\mathrm{R'_{gid,k,i}} = \mathrm{R_{gid}}^{t_{k,ni}}\mathrm{R_{gid,k,i}}$. Finally, $\mathrm{ASK_{S,gid,k}} = (\mathrm{SK_{gid,k}}, \left\{\mathrm{SK_{gid,k,i}}\right\}_{\mathrm{i} \in \mathrm{S_{gid,k}}})$ is given to the user.

Therefore, the final set of user keys $\mathrm{FK_{gid}}$ contains:

$$\mathrm{K1} = g^{r_{gid}}\mathrm{R_{gid}} \tag{9}$$

$$\mathrm{K2} = \{\mathrm{SK_{gid,k}}\} \tag{10}$$

$$\mathrm{K_3} = \{\left(\mathrm{SK_{gid,k,ii} \in S_{gid,k}}\right)\} \tag{11}$$

5.4. Decryption

When a user submits their data access request to the cloud server along with their secret keys, the decrypt algorithm is executed as follows:

Decrypt(CT, GPP, $\mathrm{FK_{gid}}$)$\rightarrow$(K): When the set of attribute keys of the user ($\mathrm{S_{gid}}$) matches any of the conjunction or sub-policy $(W,\ \rho,\ Z)$, then the symmetric key K can be retrieved by the following steps:

Step 1: Compute $CK = \prod_{i \in SAA}\mathrm{K_{2,i}}$, and choose constants $\mu_x \in Z_N$, such that $\sum_{\rho(x) \in \mathrm{S_{gid}}} \mu_x W_x = (1, 0, \ldots, 0)$. Then compute

$$e(\ CK,\ C_0) = e\left(\prod_{k=1}^{l} g^{\alpha_k}g^{r_{gid}\beta_k}\ \mathrm{R'_{gid,k}},\ g^s\right) = e(g,g)^{s\sum_{k=1}^{l}\alpha_k}e(g,g)^{r_{gid}s\sum_{k=1}^{l}\beta_k} \tag{12}$$

Step 2: For all attribute-related keys for which $\rho(x) \in \mathrm{S_{gid}}$, compute

$$\begin{gathered}\prod_{\rho(x) \in \mathrm{S_{gid}}}\ [e(C_{1,x}, K_1)e(C_{2,x}, K_{3,i})]^{\mu_x} = \\ \prod_{\rho(x) \in \mathrm{S_{gid}}}\left[\ e\left(g^{\sum_{k \in SAA}\beta_k\lambda x}.\left(g^{t_{\rho(x)}Z_{\rho(x)}}\right)^{-r_x},\ g^{r_{gid}}\mathrm{R_{gid}}\right)e\left(g^{r_x},\ T_{\rho(x)}^{Z_{\rho(x)}r_{gid}}\mathrm{R'_{gid,k,i}}\right)\right]^{\mu_x} \\ = \prod_{\rho(x) \in \mathrm{S_{gid}}}\left[\ e\left(g^{\sum_{k \in SAA}\beta_k\lambda x}, g^{r_{gid}}\right)\right]^{\mu_x} = e(g,g)^{r_{gid}s\sum_{k \epsilon SAA}\beta_k}\end{gathered} \tag{13}$$

Step 3: After dividing the result of step 1 by step 2, we obtain $e(g,g)^{s\sum_{k=1}^{l}\alpha_k}$.

Step 4:

$$C/\ e(g,g)^{s\sum_{k=1}^{l}\alpha_k} = K \cdot\ e(g,g)^{s\sum_{k \in SAA}\alpha_k}/\ e(g,g)^{s\sum_{k=1}^{l}\alpha_k} = K \tag{14}$$

The user can recover the data by using the symmetric key K.

6. Security and Performance Analysis

6.1. Security Analysis

The following complexity assumptions serve as the foundation for our security proofs: Subgroup Decision Problem for Three Primes [38,50]

Three assumptions are contained in this SDP assumption. Here Pr denotes the probability function.

Assumption 1: *Let us take a group generator* $\bar{g}$ *and consider the distribution:* $\mathbb{G}$=$(O = p_1p_2p_3, G, G_T, e) \overset{R}{\leftarrow} \bar{g}$, $g \overset{R}{\leftarrow} G_{p_1}, X_3 \overset{R}{\leftarrow} G_{p_3}, D = (\mathbb{G}, g, X_3)$, $T_1 \overset{R}{\leftarrow} G_{p_1p_2}, T_2 \overset{R}{\leftarrow} G_{p_1}$. *The advantage of an algorithm A in breaking the mentioned assumption is:*

$$SDP_Adv1_{\mathcal{G},\mathrm{A}}(\lambda) = |Pr[\,\mathrm{A}(D, T_1) = 1] - \Pr[\,\mathrm{A}(D, T_2) = 1] \tag{15}$$

Definition 7. *If for any polynomial time (PT) algorithm A,* $SDP_Adv1_{\mathcal{G},\mathcal{A}}(\lambda)$ *is a negligible function of* λ, *then* $\bar{g}$ *is said to satisfy the above-mentioned assumption 1.*

Assumption 2: *Let us take a group generator* $\bar{g}$ *and consider the distribution:* $\mathbb{G} = (O = p_1p_2p_3, G, G_T, e) \overset{R}{\leftarrow} \bar{g}, g, X_1 \overset{R}{\leftarrow} G_{p_1}, X_2, Y_2 \overset{R}{\leftarrow} G_{p_2}, X_3, Y_3 \overset{R}{\leftarrow} G_{p_3}, D = (\mathbb{G}, g, X_1X_2, X_3, Y_2Y_3)$, $T_1 \overset{R}{\leftarrow} G, T_2 \overset{R}{\leftarrow} G_{p_2p_3}$. *The advantage of an algorithm A in breaking the mentioned assumption is:*

$$SDP_Adv2_{\mathcal{G},\mathrm{A}}(\lambda) = |Pr[\mathrm{A}(D, T_1) = 1] - Pr[\mathrm{A}(D, T_2) = 1]| \tag{16}$$

Definition 8. *If for any polynomial-time (PT) algorithm A,* $SDP_Adv2_{\mathcal{G},\mathcal{A}}(\lambda)$ *is a negligible function of* λ, *then* $\bar{g}$ *is said to satisfy the above-mentioned assumption 2.*

Assumption 3: *Let us take a group generator* $\bar{g}$ *and consider the distribution:* $\mathbb{G}$=$(O = p_1p_2p_3, G, G_T, e) \overset{R}{\leftarrow} \mathcal{G}, \alpha, s \overset{R}{\leftarrow} \mathbb{Z}_N, g \overset{R}{\leftarrow} G_{p_1}, X_2, Y_2, Z_2 \overset{R}{\leftarrow} G_{p_2}, X_3 \overset{R}{\leftarrow} G_{p_3}, D$=$(\mathbb{G}, g, g^{\alpha}X_2, X_3, g^{s}Y_2, Z_2)$, $T_1 = e(g, g)^{\alpha}s, T_2 \overset{R}{\leftarrow} G_T$. *The advantage of an algorithm A in breaking the mentioned assumption is:*

$$SDP_Adv3_{\mathcal{G},\mathrm{A}}(\lambda) = |Pr[\mathrm{A}(D, T_1) = 1] - Pr[\mathrm{A}(D, T_2) = 1]| \tag{17}$$

Definition 9. *If for any polynomial-time (PT) algorithm A,* $SDP_Adv3_{\mathcal{G},\mathcal{A}}(\lambda)$ *is a negligible function of* λ, *then* $\bar{g}$ *is said to satisfy the above-mentioned Assumption 3.*

Theorem 1. *If the above three assumptions are true, then the proposed SP-MAACS scheme is fully secure according to the model presented in* Section 4.3*—Definition 6.*

Proof. We will use two terms here: ciphertext in semi-functional form (SF-CT) and key in semi-functional form (SF-Key). The terms are used in proof [16] and are not used in the construction of the scheme. We chose a random exponent $z_{k,i} \in Z_N$ for each attribute $i \in U_k$. □

Semi-functional ciphertext. To make an SF-CT, perform the following:

Let g_2 be a generator of G_{p_2}, $d \overset{R}{\leftarrow} Z_N$. ∀row $x \epsilon [l]$, let us randomly select $\xi x \in Z_N$. In addition, choose a random vector $\vec{y} \in Z_N^n$. Then, set

$$C_0 = g^s \cdot {g_2}^d$$

For each row $x \epsilon [l]$,

$$C_{1,x} = g^{\sum_{k \in SAA} \beta_k \lambda x} \cdot \left(T^{Z_{\rho(x)}}_{\rho(x)}\right)^{-r_x} \cdot {g_2}^{W_x \cdot \vec{y} + \xi x z_{\rho(x)}}$$

$$C_{2,x} = g^{r_x} \cdot {g_2}^{-\xi x}$$

Semi-functional key: There can be two types:

Type 1 SF-Key: Choose random exponents r, $\delta_k \in Z_N$ and set:

$$\mathrm{CASK_{gid}} = g^{r_{gid}} \mathrm{R_{gid}} \cdot g_2{}^{r}$$

$$CAPK_{gid} = \left(gid, \mathrm{CASK_{gid}}, \gamma_{gid}\right)$$

$$\mathrm{SK_{gid,k}} = g^{\alpha_k} g^{r_{gid}\beta_k} \mathrm{R'_{gid,k}} \cdot g_2{}^{\delta_k}$$

$$\mathrm{SK_{gid,k,i}} = T_{k,ni}{}^{r_{gid}} \mathrm{R'_{gid,k,i}} \cdot g_2{}^{r z_{\mathrm{k,i}}}$$

Type 2 SF-Key:

$$\mathrm{CASK_{gid}} = g^{r_{gid}} \mathrm{R_{gid}}$$

$$CAPK_{gid} = \left(gid, \mathrm{CASK_{gid}}, \gamma_{gid}\right)$$

$$\mathrm{SK_{gid,k}} = g^{\alpha_k} g^{r_{gid}\beta_k} \mathrm{R'_{gid,k}} \cdot g_2{}^{\delta_k}$$

$$\mathrm{SK_{gid,k,i}} = T_{k,ni}{}^{r_{gid}} \mathrm{R'_{gid,k,i}}$$

If we use a regular key to decipher an SF-CT or an SF-Key to decipher a regular ciphertext, we can correctly calculate $\left(\prod_{k \in SAA} e(g,g)^{\alpha_k s}\right)$. However, if we try to use an SF-Key to decipher a semi-functional CT, it will give us an extra thing: $e(g_2, g_2)^{c \sum_{k \in SAA} \delta_k - r y_1}$, where y_1 is the first coordinate of the vector $\vec{y}$.

The adaptive security of the scheme from three assumptions (Assumption no. 1, 2, 3), can be confirmed using a sequence of games shown in the appendix of [38].

6.2. Performance Analysis

As shown in Table 1, our scheme is compared with prominent multi-authority schemes, single-authority privacy-preserving schemes, and some privacy-preserving multi-authority schemes. The salient features of our scheme are also highlighted in Sections 1 and 2. Table 2 shows the comparison of storage overhead and encryption and decryption computation costs. Table 3 presents a summary of the notations that are used in the comparison.

Table 2. Storage and computation cost comparison.

Scheme	Storage Overhead			Computation Cost	
	Public Key	User's Secret Key	Ciphertext	Encryption	Decryption
[28]	$\lvert G_T\rvert + (n+4)\lvert G\rvert$	$(\lvert A_U\rvert + 2)\lvert G\rvert$	$(4\lvert A_C\rvert + 2)\lvert G\rvert + 2\lvert G_T\rvert$	$(7 \lvert A_C\rvert + 4)$ E	$(2\lvert I\rvert + 4)$P + $(4\lvert I\rvert)$E
[38]	$\lvert G_T\rvert + 4\lvert G\rvert$	$(\lvert A_U\rvert + 2)\lvert G\rvert$	$(3\lvert A_C\rvert + 2)\lvert G\rvert + 2\lvert G_T\rvert$	$(7 \lvert A_C\rvert + 4)$ E	$(2\lvert I\rvert + 3)$P + $(3\lvert I\rvert)$E
[16]	$n\lvert G_T\rvert + n\lvert G\rvert$	$(\lvert A_U\rvert)\lvert G\rvert$	$(2\lvert A_C\rvert)\lvert G\rvert + (\lvert A_C\rvert + 1)\lvert G_T\rvert$	$(5 \lvert A_C\rvert + 1)$ E	$(2\lvert I\rvert)$P + $(\lvert I\rvert)$E
[23]	$n_a\lvert G_T\rvert + (n + n_a + 1)\lvert G\rvert$	$(n_a + \lvert A_U\rvert + 2)\lvert G\rvert$	$(2\lvert A_C\rvert + 1)\lvert G\rvert + \lvert G_T\rvert$	$(3\lvert A_C\rvert + 2)$E	$(2\lvert I\rvert + 1)$P + $(\lvert I\rvert + 1)$E
[26]	$n\lvert G_T\rvert + (n+1)\lvert G\rvert$	$(\lvert A_U\rvert + 1)\lvert G\rvert$	$(2\lvert A_C\rvert)\lvert G\rvert + (\lvert A_C\rvert + 1)\lvert G_T\rvert$	$(5 \lvert A_C\rvert + 1)$ E	$(2\lvert I\rvert)$P + $(3\lvert I\rvert)$E
[35]	$n\lvert G_T\rvert + n\lvert G\rvert$	$(2\lvert A_U\rvert)\lvert G\rvert$	$(2\lvert A_C\rvert + 1)\lvert G\rvert + (\lvert A_C\rvert + 1)\lvert G_T\rvert$	$(5 \lvert A_C\rvert + 1)$ E	$(2\lvert I\rvert + 1)$P + $\lvert I\rvert$E
[39]	$n_a\lvert G_T\rvert + (n + n_v)\lvert G\rvert$	$(n_a + \lvert A_U\rvert)\lvert G\rvert$	$(2\lvert A_C\rvert + 1)\lvert G\rvert + \lvert G_T\rvert$	$(5\lvert A_C\rvert + 1)$E	$(3\lvert I\rvert)$P + $\lvert I\rvert$E
[Ours]	$n_a\lvert G_T\rvert + (n + n_v + 1)\lvert G\rvert$	$(n_a + \lvert A_U\rvert + 2)\lvert G\rvert$	$(2\lvert A_C\rvert + 1)\lvert G\rvert + \lvert G_T\rvert$	$(3\lvert A_C\rvert + 2)$E	$(2\lvert I\rvert + 1)$P + $\lvert I\rvert$ E

Table 3. Summary of notations.

Notation	Description
$\|G\|$	No. of bits needed to represent an element in group G
$\|G_T\|$	No. of bits needed to represent an element in group G_T
$\|\mathbf{I}\|$	No. of attributes of satisfying set
n	The size of universe of attributes
A_C	Attribute set used in encryption
A_U	User's attribute set
E	One exponential operation
P	One pairing operation
n_a	Count of AAs in the system
n_v	Number of values all the attributes in the system may have (an attribute may have multiple values)

From the above numerical performance analysis, it can be observed that the key generation time of our scheme and schemes [23,39] are comparable, while other schemes have shorter key generation times. The reason for this is that either they are single-authority or not provide privacy preservation features. The numerical decryption time of our scheme is less than that of the others, as the number of exponentiation operations performed is less.

6.3. Implementation Result

Through the characteristic comparison presented in Table 1, our scheme SP-MAACS has been shown to be better than other schemes in terms of features attained. We implemented our scheme and the fully secure decentralized CP-ABE scheme [26] and assessed the performance. The authors of fully secure decentralized CP-ABE presented two constructions [26] and demonstrated them to be secure under the standard model. Their first construction used a composite-order bilinear group and was confirmed to be fully secure by taking static assumptions. The scheme applies to any monotone access structure.

6.3.1. Implementation Environment

The SP-MAACS scheme and the fully secure decentralized CP-ABE scheme [26] were implemented using the well-developed and robust JPBC library [51]. We used the Eclipse IDE to implement the simulation code and the code was written in Java. The tests were conducted on a laptop running Windows 10 (64-bit) and equipped with a 2.50 GHz Intel (R) i5-3210M processor and 4 gigabytes of RAM.

6.3.2. Implementation Setup

ABE schemes can be implemented using pairing-based cryptography. The Java Pairing Based Cryptography JPBC library [51] is extensively used in developing cryptographic solutions. While configuring JPBC, the following .jar files are included: jpbc-pbc-v2.0.0-m.jar, jpbc-plaf-v2.0.0-m.jar, jpbc-api-v2.0.0-m.jar, jpbc-crypto-v2.0.0-m.jar, and bcprov-jdk16-1.46.jar. JPBC includes various packages such as api, util, pairing, field, etc. Some interfaces for pairing functions, finite fields, and elliptic curves are provided by package api. Package util offers support for mathematical operations and other functions. Package pairing and field are the concrete realizations of the interfaces offered by package api. JPBC supports a variety of elliptic curve types. Type A, Type A1, Type D, Type E, Type F, and Type G are included. This experiment was conducted on the group of elliptic curves of Type A1. Their order is the product of three primes of length 517 bits. In the global setup method, while performing setup for the central authority, the Boneh–Lynn–Shacham signature scheme, called the BLS signature, was implemented. A BLS signature enables a user to validate the authenticity of a signer. Signatures are created as elements in elliptic

curve groups and verified using a pairing function. The Junit testing framework was used for testing the implemented classes.

6.3.3. System Setup

Exponentiations and pairing operations in the above algorithms for the encryption and decryption process account for a large portion of the computational overhead of CP-ABE systems; thus, we analyzed the encryption and decryption cost of our scheme. For the analysis purpose, we assumed that in our system, five attribute authorities are responsible for attribute management. In addition, it was assumed that each authority manages five attributes. Each attribute may have any one of the possible attribute values. We also implemented a secure signature scheme, which was used by CA to sign the user key, and AA use verified the algorithm to perform the verification of the key.

6.3.4. Result Analysis

Figure 3 illustrates the computation time of the encryption, key generation, and decryption algorithms. Each algorithm's experimental outcome was the mean of 10 independent runs. Figure 3a presents a graph between the encryption time and the attribute count in the access control policy. We took several attributes on the X-axis in the range of 5 attributes to 25 attributes. We have already mentioned that our scheme considers the DNF access structure and the scheme [26] uses a traditional AND-OR access structure. Typically, when a data owner decides on an access policy, they start to build it by combining many possibilities using the "OR" gate. This standard method of specifying access policy corresponds to the DNF structure used by our approach. The time taken for encryption in our scheme is higher because we perform encryption for every access sub-structure clause. This, in turn, increases the number of ciphertext components. It can be observed that the encryption activity is less frequent. Due to the vast computational power and storage capacities of the cloud, increased encryption time and storage needs are acceptable.

Figure 3b shows the key generation time versus attribute count in the satisfying set. In SP-MAACS, the key generation time is higher, as the user secret key contains the identity and attribute-value-related key components in addition to the authority-related keys. We use identity and attribute-value-related keys to provide collision resistance and privacy preservation features. The central authority CA of the system registers the user and issues them identity-related keys, and are the keys are issued to the user by the CA (shown in the construction of the scheme). The user may have multiple values for their attributes. The attribute authority AA issues both authority-related and attribute-value-specific keys to the user. Scheme [26], on the other hand, only considers keys provided by the attribute authority. Therefore, we compromise key generation time to deliver more functionality. Figure 3c shows a graph between the decryption times and attribute count. In Figure 3c, attributes present in policy and attribute count in the satisfying sets are shown on the dual X-axis. While performing decryption, the user can determine the smallest satisfying set out of the policy defined by the owner of the data. In general, we can assume that on average 50% of the attributes make a satisfying subset out of the total attributes present in the policy. For example, if an access control policy contains 20 attributes in it, then in general the AND-OR combination requires that the user should carry on average 10 attributes for satisfying it or the policy require only 10 attributes to fulfill it. The DNF clause, which matches the attribute and its value in the user-supplied keys, is only utilized during the decryption process. Since there is no requirement for attribute matching with the whole access structure, our scheme takes less decryption time and improves decryption efficiency in comparison to the scheme [26]. This improvement in decryption time satisfies the major requirement of healthcare data sharing, where a doctor wants to access a patient's EHR data quickly in a life-critical situation.

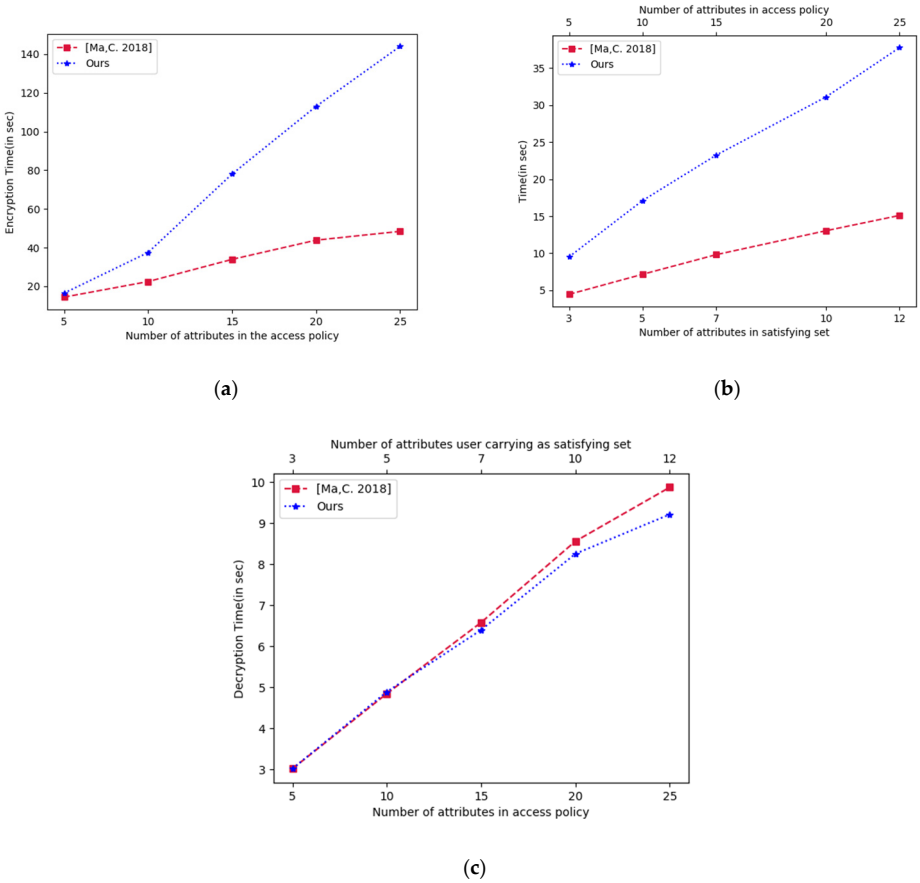


Figure 3. (**a**) Encryption time, (**b**) attribute key generation time, (**c**) decryption time.

In summary, our scheme simultaneously provides features of multi-authority, privacy preservation, efficient decryption, and adaptive security. The increase in storage cost and encryption time is affordable, as cloud storage is available at a very nominal cost and encryption activity is performed less frequently. The users may ask for the download and decryption of the ciphertext randomly as per their requirement, so it is a frequent process. Thus, reducing the decryption cost is beneficial.

7. Conclusions

Today, the cloud is the most obvious data-sharing platform for the healthcare sector, and ABE schemes can be used to provide access control on outsourced EHR data in encrypted form. In order to share data on cloud storage servers, this article suggests the SP-MAACS scheme, a completely secure and privacy-preserving multi-authority access control system for cloud-based healthcare data sharing. Data owners may now freely share their data with all users in both open and closed domains. This makes the system scalable and adaptable. The partially hidden access policy protects user as well as data owner privacy. Our implementation results demonstrate that the scheme achieves an improvement in decryption cost despite the scheme being privacy-preserving and providing adaptive security under the standard model. The efficiency of decryption can be further increased by outsourcing the decryption to proxy servers in the future. Healthcare data management and privacy protection are currently one of the most active blockchain research areas. Combining the proposed control scheme with blockchain technology could improve security, privacy, and audibility.

Author Contributions: Conceptualization, R.G., P.K. and N.D.; methodology, R.G., P.K. and N.D.; software, R.G. and K.S.S.; validation, P.K., N.D. and G.M.; formal analysis, P.K.; investigation, N.Z.J., M.M., N.S.A. and M.A.A.; data curation, R.G.; writing—original draft preparation, R.G. and N.D.; writing—review and editing, G.M. and K.S.S.; visualization, R.G. and N.D.; supervision, G.M., K.S.S. and N.Z.J.; funding acquisition, M.M., N.S.A. and M.A.A. All authors have read and agreed to the published version of the manuscript.

Funding: Researchers Supporting Project number (RSPD2023R521), King Saud University, Riyadh, Saudi Arabia.

Institutional Review Board Statement: Ethical approval was not required for this study. Human or animal subject data were not used in this study.

Informed Consent Statement: Not applicable.

Data Availability Statement: Available on request.

Conflicts of Interest: The authors declare no conflict of interest.

References

1. Nasiri, S.; Sadoughi, F.; Tadayon, M.H.; Dehnad, A. Security requirements of internet of things-based healthcare system: A survey study. *Acta Inform. Med.* **2019**, *27*, 253. [CrossRef]
2. Kanwal, T.; Anjum, A.; Khan, A. Privacy preservation in e-health cloud: Taxonomy, privacy requirements, feasibility analysis, and opportunities. *Clust. Comput.* **2021**, *24*, 293–317. [CrossRef]
3. Goyal, V.; Pandey, O.; Sahai, A.; Waters, B. Attribute-based encryption for fine-grained access control of encrypted data. In Proceedings of the 13th ACM Conference on Computer and Communications Security, Alexandria, VA, USA, 30 October–3 November 2006; pp. 89–98.
4. Bethencourt, J.; Sahai, A.; Waters, B. Ciphertext-policy attribute-based encryption. In Proceedings of the 2007 IEEE Symposium on Security and Privacy (SP'07), Berkeley, CA, USA, 20–23 May 2007; pp. 321–334. [CrossRef]
5. Hur, J. Improving security and efficiency in attribute-based data sharing. *IEEE Trans. Knowl. Data Eng.* **2011**, *25*, 2271–2282. [CrossRef]
6. Zhang, R.; Ma, H.; Lu, Y. Fine-grained access control system based on fully outsourced attribute-based encryption. *J. Syst. Softw.* **2017**, *125*, 344–353. [CrossRef]
7. Xue, K.; Chen, W.; Li, W.; Hong, J.; Hong, P. Combining data owner-side and cloud-side access control for encrypted cloud storage. *IEEE Trans. Inf. Forensics Secur.* **2018**, *13*, 2062–2074. [CrossRef]
8. Li, Q.; Zhang, Y.; Zhang, T.; Huang, H.; He, Y.; Xiong, J. HTAC: Fine-grained policy-hiding and traceable access control in mHealth. *IEEE Access* **2020**, *8*, 123430–1234039. [CrossRef]
9. Cheung, L.; Newport, C. Provably secure ciphertext policy ABE. In Proceedings of the 14th ACM Conference on Computer and Communications Security, Alexandria, VA, USA, 2 November–31 October 2007; pp. 456–465.
10. Gupta, R.; Kanungo, P.; Dagdee, N. Sharing Organizational Data Outside Its Domain Using ABE in Cloud Environment. In *International Conference on Advanced Computing Networking and Informatics 2019*; Springer: Singapore, 2019; pp. 69–77.
11. Sahai, A.; Waters, B. Fuzzy identity-based encryption. In Proceedings of the Annual International Conference on the Theory and Applications of Cryptographic Techniques, Aarhus, Denmark, 22–26 May 2005; Springer: Berlin/Heidelberg, Germany, 2005; pp. 457–473.
12. Waters, B. Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization. In Proceedings of the Public Key Cryptography–PKC 2011: 14th International Conference on Practice and Theory in Public Key Cryptography, Taormina, Italy, 6–9 March 2011; Springer: Berlin/Heidelberg, 2011; pp. 53–70.
13. Chase, M. Multi-authority attribute based encryption. In Proceedings of the Theory of Cryptography Conference, Amsterdam, The Netherlands, 21–24 February 2007; Springer: Berlin/Heidelberg, Germany, 2007; pp. 515–534.
14. Chase, M.; Chow, S.S. Improving privacy and security in multi-authority attribute-based encryption. In Proceedings of the 16th ACM Conference on Computer and Communications Security, Chicago, IL, USA, 9–13 November 2009; pp. 121–130.
15. Liu, Z.; Cao, Z.; Huang, Q.; Wong, D.S.; Yuen, T.H. Fully secure multi-authority ciphertext-policy attribute-based encryption without random oracles. In Proceedings of the European Symposium on Research in Computer Security, Leuven, Belgium, 12–14 September 2011; Springer: Berlin/Heidelberg, Germany, 2011; pp. 278–297.
16. Lewko, A.; Waters, B. Decentralizing attribute-based encryption. In Proceedings of the Advances in Cryptology–EUROCRYPT 2011: 30th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tallinn, Estonia, 15–19 May 2011; Springer: Berlin/Heidelberg, Germany, 2011; pp. 568–588.
17. Li, M.; Yu, S.; Ren, K.; Lou, W. Securing personal health records in cloud computing: Patient-centric and fine-grained data access control in multi-owner settings. In Proceedings of the International Conference on Security and Privacy in Communication Systems, Singapore, 7–9 September 2010; Springer: Berlin/Heidelberg, Germany, 2010; pp. 89–106.

18. Ibraimi, L.; Asim, M.; Petković, M. Secure management of personal health records by applying attribute-based encryption. In Proceedings of the 6th International Workshop on Wearable, Micro, and Nano Technologies for Personalized Health, Oslo, Norway, 24–26 June 2009; pp. 71–74.
19. Ruj, S.; Stojmenovic, M.; Nayak, A. Decentralized access control with anonymous authentication of data stored in clouds. *IEEE Trans. Parallel Distrib. Syst.* **2013**, *25*, 384–394. [CrossRef]
20. Yang, K.; Jia, X.; Ren, K.; Zhang, B.; Xie, R. DAC-MACS: Effective data access control for multiauthority cloud storage systems. *IEEE Trans. Inf. Forensics Secur.* **2013**, *8*, 1790–1801. [CrossRef]
21. Gardiyawasam Pussewalage, H.S.; Oleshchuk, V.A. A distributed multi-authority attribute based encryption scheme for secure sharing of personal health records. In Proceedings of the 22nd ACM on Symposium on Access Control Models and Technologies, Indianapolis, IN, USA, 21–23 June 2017; pp. 255–262.
22. Ibraimi, L.; Tang, Q.; Hartel, P.; Jonker, W. Efficient and provable secure ciphertext-policy attribute-based encryption schemes. In Proceedings of the International Conference on Information Security Practice and Experience, Xi'an, China, 13–15 April 2009; Springer: Berlin/Heidelberg, Germany, 2009; pp. 1–12.
23. Li, Q.; Ma, J.; Li, R.; Liu, X.; Xiong, J.; Chen, D. Secure, efficient and revocable multi-authority access control system in cloud storage. *Comput. Secur.* **2016**, *59*, 45–59. [CrossRef]
24. Hu, S.; Li, J.; Lu, Y.; Zhang, Y. Constant Ciphertext Size Multi-Authority Attribute-based Scheme without Key Escrow. *J. Internet Technol.* **2020**, *21*, 1647–1657.
25. Li, Q.; Zhu, H.; Xiong, J.; Mo, R.; Ying, Z.; Wang, H. Fine-grained multi-authority access control in IoT-enabled mHealth. *Ann. Telecommun.* **2019**, *74*, 389–400. [CrossRef]
26. Ma, C.; Ge, A.; Zhang, J. Fully secure decentralized ciphertext-policy attribute-based encryption in standard model. In Proceedings of the International Conference on Information Security and Cryptology, Seoul, Republic of Korea, 28–30 November 2018; Springer: Cham, Switzerland, 2018; pp. 427–447.
27. Nishide, T.; Yoneyama, K.; Ohta, K. Attribute-based encryption with partially hidden encryptor-specified access structures. In Proceedings of the International Conference on Applied Cryptography and Network Security, New York, NY, USA, 3–6 June 2008; Springer: Berlin/Heidelberg, Germany, 2008; pp. 111–129.
28. Lai, J.; Deng, R.H.; Li, Y. Expressive CP-ABE with partially hidden access structures. In Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security, Seoul, Republic of Korea, 2–4 May 2012; pp. 18–19.
29. Li, J.; Huang, Q.; Chen, X.; Chow, S.S.; Wong, D.S.; Xie, D. Multi-authority ciphertext-policy attribute-based encryption with accountability. In Proceedings of the 6th ACM Symposium on Information, Computer and Communications Security, Hong Kong, China, 22–24 March 2011; pp. 386–390.
30. Han, J.; Susilo, W.; Mu, Y.; Zhou, J.; Au, M.H. Improving privacy and security in decentralized ciphertext-policy attribute-based encryption. *IEEE Trans. Inf. Forensics Secur.* **2014**, *10*, 665–678.
31. Pedersen, T.P. Non-interactive and information-theoretic secure verifiable secret sharing. In Proceedings of the Annual International Cryptology Conference, Santa Barbara, CA, USA, 16–20 August 1992; Springer: Berlin/Heidelberg, Germany, 1992; pp. 129–140.
32. Zhang, Y.; Chen, X.; Li, J.; Wong, D.S.; Li, H. Anonymous attribute-based encryption supporting efficient decryption test. In Proceedings of the 8th ACM SIGSAC Symposium on Information, Computer and Communications Security, Hangzhou, China, 8–10 May 2013; pp. 511–516.
33. Chen, J.; Ma, H. Privacy-preserving decentralized access control for cloud storage systems. In Proceedings of the 2014 IEEE 7th International Conference on Cloud Computing, Anchorage, AK, USA, 27 June–2 July 2014; pp. 506–513.
34. Li, J.; Li, N. OACerts: Oblivious attribute certificates. *IEEE Trans. Dependable Secur. Comput.* **2006**, *3*, 340–352. [CrossRef]
35. Zhong, H.; Zhu, W.; Xu, Y.; Cui, J. Multi-authority attribute-based encryption access control scheme with policy hidden for cloud storage. *Soft Comput.* **2018**, *22*, 243–251. [CrossRef]
36. Yang, K.; Han, Q.; Li, H.; Zheng, K.; Su, Z.; Shen, X. An efficient and fine-grained big data access control scheme with privacy-preserving policy. *IEEE Internet Things J.* **2016**, *4*, 563–571. [CrossRef]
37. Ying, Z.; Wei, L.; Li, Q.; Liu, X.; Cui, J. A lightweight policy preserving EHR sharing scheme in the cloud. *IEEE Access* **2018**, *6*, 53698–53708. [CrossRef]
38. Zhang, Y.; Zheng, D.; Deng, R.H. Security and privacy in smart health: Efficient policy-hiding attribute-based access control. *IEEE Internet Things J.* **2018**, *5*, 2130–2145. [CrossRef]
39. Yan, X.; Ni, H.; Liu, Y.; Han, D. Privacy-preserving multi-authority attribute-based encryption with dynamic policy updating in PHR. *Comput. Sci. Inf. Syst.* **2019**, *16*, 831–847. [CrossRef]
40. Belguith, S.; Kaaniche, N.; Laurent, M.; Jemai, A.; Attia, R. Phoabe: Securely outsourcing multi-authority attribute based encryption with policy hidden for cloud assisted iot. *Comput. Netw.* **2018**, *133*, 141–156. [CrossRef]
41. Zhang, L.; Hu, G.; Mu, Y.; Rezaeibagha, F. Hidden ciphertext policy attribute-based encryption with fast decryption for personal health record system. *IEEE Access* **2019**, *7*, 33202–33213. [CrossRef]
42. Chinnasamy, P.; Deepalakshmi, P.; Dutta, A.K.; You, J.; Joshi, G.P. Ciphertext-Policy Attribute-Based Encryption for Cloud Storage: Toward Data Privacy and Authentication in AI-Enabled IoT System. *Mathematics* **2021**, *10*, 68. [CrossRef]
43. Sing, R.; Bhoi, S.K.; Panigrahi, N.; Sahoo, K.S.; Jhanjhi, N.; AlZain, M.A. A Whale Optimization Algorithm Based Resource Allocation Scheme for Cloud-Fog Based IoT Applications. *Electronics* **2022**, *11*, 3207. [CrossRef]

44. Pande, S.K.; Panda, S.K.; Das, S.; Sahoo, K.S.; Luhach, A.K.; Jhanjhi, N.Z.; Alroobaea, R.; Sivanesan, S. A resource management algorithm for virtual machine migration in vehicular cloud computing. *Comput. Mater. Contin.* **2021**, *67*, 2647–2663.
45. Najafi, A.; Bayat, M.; Haj Seyyed Javadi, H. Privacy Preserving Attribute-Based Encryption with Conjunctive Keyword Search for E-health Records in Cloud. *ISC Int. J. Inf. Secur.* **2021**, *13*, 87–100.
46. Lee, S.; Kim, J.; Kwon, Y.; Kim, T.; Cho, S. Privacy Preservation in Patient Information Exchange Systems Based on Blockchain: System Design Study. *J. Med. Internet Res.* **2022**, *24*, e29108. [CrossRef] [PubMed]
47. Crampton, J.; Pinto, A. Attribute-based encryption for access control using elementary operations. In Proceedings of the 2014 IEEE 27th Computer Security Foundations Symposium, Vienna, Austria, 19–22 July 2014; pp. 125–139.
48. Yan, X.; He, G.; Yu, J.; Tang, Y.; Zhao, M. Offline/online outsourced attribute-based encryption with partial policy hidden for the internet of things. *J. Sens.* **2020**, *2020*, 1–11. [CrossRef]
49. Boneh, D.; Goh, E.J.; Nissim, K. Evaluating 2-DNF formulas on ciphertexts. In *Theory of Cryptography Conference, Cambridge, MA, USA, 10–12 February 2005*; Springer: Berlin/Heidelberg, Germany, 2005; pp. 325–341.
50. Lewko, A.; Okamoto, T.; Sahai, A.; Takashima, K.; Waters, B. Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption. In Proceedings of the Annual International Conference on the Theory and Applications of Cryptographic Techniques, Monaco and Nice, France, 30 May–3 June 2010; Springer: Berlin/Heidelberg, Germany, 2010; pp. 62–91.
51. De Caro, A.; Iovino, V. jPBC: Java pairing based cryptography. In Proceedings of the 2011 IEEE Symposium on Computers and Communications (ISCC), Kerkyra, Greece, 28 June–1 July 2011; pp. 850–855.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.